

Character Classes and Bracket Expressions

A bracket expression is a list of characters enclosed by '[' and ']'. It matches any single character in that list. If the first character of the list is the caret '^', then it matches any character not in the list, and it is unspecified whether it matches an encoding error. For example, the regular expression '[0123456789]' matches any single digit, whereas '[^()]' matches any single character that is not an opening or closing parenthesis, and might or might not match an encoding error.

Within a bracket expression, a range expression consists of two characters separated by a hyphen. In the default C locale, it matches any single character that appears between the two characters in ASCII order, inclusive. For example, '[a-d]' is equivalent to '[abcd]'. In other locales the behavior is unspecified: '[a-d]' might be equivalent to '[abcd]' or '[aBbCcDd]' or some other bracket expression, or it might fail to match any character, or the set of characters that it matches might be erratic, or it might be invalid. To obtain the traditional interpretation of bracket expressions, you can use the 'C' locale by setting the LC_ALL environment variable to the value 'C'.

Finally, certain named classes of characters are predefined within bracket expressions, as follows. Their interpretation depends on the LC_CTYPE locale; for example, '[[:alnum:]]' means the character class of numbers and letters in the current locale.

Predefined Character Classes

- **[[:alnum:]]** Alphanumeric characters: '[[:alpha:]]' and '[[:digit:]]'; in the 'C' locale and ASCII character encoding, this is the same as '[0-9A-Za-z]'.
- **[[:alpha:]]** Alphabetic characters: '[[:lower:]]' and '[[:upper:]]'; in the 'C' locale and ASCII character encoding, this is the same as '[A-Za-z]'.
- **[[:blank:]]** Blank characters: space and tab.
- **[[:cntrl:]]** Control characters. In ASCII, these characters have octal codes 000 through 037, and 177 (DEL). In other character sets, these are the equivalent characters, if any.
- **[[:digit:]]** Digits: 0 1 2 3 4 5 6 7 8 9.
- **[[:graph:]]** Graphical characters: '[[:alnum:]]' and '[[:punct:]]'.
- **[[:lower:]]** Lower-case letters; in the 'C' locale and ASCII character encoding, this is a b c d e f g h i j k l m n o p q r s t u v w x y z.
- **[[:print:]]** Printable characters: '[[:alnum:]]', '[[:punct:]]', and space.
- **[[:punct:]]** Punctuation characters; in the 'C' locale and ASCII character encoding, this is ! " # \$ % & ' () * + , - . / : ; < = > ? @ [\] ^ _ ` { | } ~.
- **[[:space:]]** Space characters: in the 'C' locale, this is tab, newline, vertical tab, form feed, carriage return, and space. See Usage, for more discussion of matching newlines.
- **[[:upper:]]** Upper-case letters: in the 'C' locale and ASCII character encoding, this is A B C D E F G H I J K L M N O P Q R S T U V W X Y Z.
- **[[:xdigit:]]** Hexadecimal digits: 0 1 2 3 4 5 6 7 8 9 A B C D E F a b c d e f.

Note that the brackets in these class names are part of the symbolic names, and must be included in addition to the brackets delimiting the bracket expression.

If you mistakenly omit the outer brackets, and search for say, '[[:upper:]]', GNU grep prints a diagnostic and exits with status 2, on the assumption that you did not intend to search for the regular expression '[[:epru]]'.

Special Characters within Bracket Expressions

Special characters lose their special meaning inside bracket expressions.

- **]** ends the bracket expression if it's not the first list item. So, if you want to make the ']' character a list item, you must put it first.
- **[** represents the open collating symbol.

- `.]` represents the close collating symbol.
- `[=` represents the open equivalence class.
- `=]` represents the close equivalence class.
- `[:` represents the open character class symbol, and should be followed by a valid character class name.
- `:]` represents the close character class symbol.
- `-` represents the range if it's not first or last in a list or the ending point of a range. To make the '-' a list item, it is best to put it last.
- `^` represents the characters not in the list. If you want to make the '^' character a list item, place it anywhere but first.